Open Access | https://doi.org/10.25080/ejwp2755



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

OXA: An Open Exchange Architecture for Modular and Composable Scientific Content

Rowan Cockett[1,2,3], **Nokome Bentley**[4], **Monica Granados**[5], **Franklin Koch**[2,3], **Michael Markie**[6], **Steve Purves**[2,3], **Carlos Scheidegger**[7], **Paul Shannon**[6], **Gabriel Stein**[8], and **Tracy Teal**[8]

[1]Continuous Science Foundation, [2]Curvenote Inc., [3]Project Jupyter, [4]Stencila, [5]Creative Commons, [6]eLife Pathways, [7]Posit, PBC, [8]openRxiv

Abstract

Scientific communication remains trapped in a paper-shaped container. The dominant unit of exchange — a static, narrative PDF backed by Journal Article Tag Suite (JATS) XML — was designed to describe finished, print-shaped documents. It is poorly suited to modern research, in which the data, code, protocols, computational notebooks, and interactive results that constitute the actual evidence are relegated to hard-to-access supplementary files. This paper introduces the Open Exchange Architecture (OXA), an emerging, community-driven specification that represents scientific documents and their components as structured, typed, and uniquely addressable JSON objects. OXA is designed to enable exchange, interoperability, and long-term preservation while remaining compatible with modern web and data standards. We describe the technical architecture of OXA — a typed node model with `children` arrays inspired by `unified.js` and the Pandoc abstract syntax tree (AST) — and trace its provenance through JATS, the Stencila schema for executable documents, Curvenote's connected publishing model, and the document pipelines of Pandoc, MyST Markdown, and Quarto. We report on the first large-scale implementation of OXA: a Curvenote–openRxiv partnership that translated the bioRxiv JATS archive into an early version of OXA to power the openRxiv Labs "Curvenote Reader" experience. OXA is stewarded by the Continuous Science Foundation and governed through an open Request for Comments (RFC) process.

Keywords modular science, open standards, reproducibility, executable documents, scientific publishing, JATS

Published Jul 13, 2026

Correspondence to
Rowan Cockett
rowan@curvenote.com

Open Access

Copyright © 2026 Cockett *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction: Toward Modular Science

The way scientists conduct research has changed profoundly over the past two decades. Data are larger, analyses are computational, and collaboration is global and continuous. Yet the way we communicate research has barely moved: the canonical output is still a static document modeled on the printed page. Science is, in effect, trapped in a paper-shaped box, where content is static and narrative-driven, and the assets around the research — the data, code, protocols, and notebooks — are relegated to supplementary ZIP files that are hard to access. The reviews and credit attached to research accrue to the container itself rather than to the components beneath it.

The Open Exchange Architecture (OXA) is a response to this mismatch. It reconceives a scientific document not as a monolithic file but as a tree of modular, composable components — figures, notebooks, methods, datasets, equations, and computations — each of which can be directly referenced, reused, and recombined across the open-science ecosystem. This is the central idea of *modular science*: that modularity should emerge from how content is

packaged, so that a figure produced in a computational notebook can be cited, embedded, attributed, and licensed as a first-class object rather than copy-pasted as a flattened screenshot.

This shift matters now for two converging reasons. First, computational science increasingly produces evidence (interactive figures, multi-gigabyte datasets, executable notebooks) that simply cannot be expressed in a PDF. Second, machine-assisted discovery and AI systems are well complemented when they can also operate over structured, interoperable components, and relational metadata [1], [2]. A modular, structured substrate is the precondition for both human and machine reuse, and it is the principle at the core of OXA.

The challenge is how we get from where we are, a static narrative document, to where we want to go, science communicated through the modular elements of science, without first requiring that scientists share their work in completely new ways or changed incentive structures. This has consistently been a sticking point during shifts towards open science practices. The burden is placed on already over-busy researchers who don't necessarily have the skills or time to do this additional work. OXA addresses this through its positioning as an exchange format. Existing articles can be decomposed into modular elements and what's then possible with these elements can be demonstrated immediately, both not requiring new work from researchers and showing what's possible when we can communicate science in new ways, so can motivate why a researcher may want to work differently. Examples such as the openRxiv and Curvenote Reader partnership described below demonstrate this approach. Additionally, OXA provides a pathway to sharing science in new ways. A researcher can begin to layer in sharing notebooks, interactive figures, data and code through the process of publishing an article. OXA helps us build the *bridge* from where we are to where we want to go.

This paper is itself written in MyST Markdown [3] and submitted through the SciPy Proceedings toolchain, which since 2024 has been built on Curvenote [4]. That toolchain is part of the same lineage we describe below, and OXA can be understood as a generalization and community standardization of the structured document model specifically for scientific communication that already underlies these tools.

2. From modular software to modular science

The SciPy community already lives in a world that is not how most of the rest of science yet operates. SciPy [5], NumPy [6], pandas [7], [8], Matplotlib [9], and the libraries built on them are modular *and* composable. A user can `import` what they need, build something, and the result inherits the trustworthiness of the modular components, composing them into something new. Progress is multiplicative because integration is expected and relatively straightforward. However, the papers we write about that work are not modular or composable. Papers are currently monolithic and integration-hostile, everything gets trapped in a paper-shaped box. The figure cannot be pulled out of or cited independently of the paper, the data cannot be queried from the paper, and the code cannot be run against the data without re-implementing context the paper only alluded to. To reach this *composable* potential, we need to be able to share about science in the same *modular* ways we share code.

It is worth being precise about a distinction that is often blurred in conversations about open science - *disaggregation* and *modularity*. *Disaggregation* is placing each research component in a different, type-appropriate repository. *Modularity* is dividing something into separate modules or components that can then be recombined. While *disaggregation* does achieve the goal of separating components, it does not necessarily easily allow for reuse or recombination. Therefore it's important to note that *disaggregation* and *modularity* are

not the same thing. The push, over the past two decades, to unbundle the scientific paper (separating preprints from peer review, peer review from venue, venue from data hosting, data hosting from code) has been focused on *disaggregation* which is an important step, especially for appropriate archiving and curation. However, disaggregation is not, on its own, the goal. The goal is **ecosystem composability**: research products whose components can be found, reused, recombined, and extended without losing their integrity or provenance Table 1.

Table 1. *Two waves of modular science.*

Dimension	First wave	Second wave
Core philosophy	Break apart to share	Package and compose
Researcher action	File separate deposits	Work natively, modularity emerges
Output state	Disconnected pieces	Connected, navigable graphs
Provenance	Lost or manually added	Inherited and travels with object
System result	Sprawl of scientific components, loosely connected	Integrated system with provenance

Modular science is the principle that scientific outputs can become as composable as scientific software, given the right standards, identifiers, tooling, and packaging infrastructure. The emerging primitives are visible: `import figure from paper` is no longer a metaphor. With component-level identifiers and a structured-document standard, a figure published in one preprint can be embedded, with full provenance, in a review article, a textbook, a grant proposal, another lab's analysis notebook, or an AI chat. With the right standards the figure stays alive; the data underneath it stays addressable; updates propagate; attribution is automatic.

3. Background

3.1. JATS and the print-shaped record

The Journal Article Tag Suite (JATS) is the de facto open standard for the scholarly version of record. It is an ANSI/NISO standard (Z39.96) and, through PubMed Central, Crossref, and indexers, underpins the machine-readable archive of the literature [10]. JATS has been enormously successful at what it was designed to do: provide a common XML vocabulary in which publishers and archives can exchange finished journal content. The scale is substantial: Crossref hosts over 96 million works with a full-text/text-mining link (96,294,821 of 134,048,223 total works, queried from the Crossref dataset via Alexandria3k [11]), the large majority of which are XML.

However, JATS carries the assumptions of the medium it was built for. It describes static, finished documents. Several well-documented friction points recur:

- **Cost and tooling.** Producing high-quality JATS requires either hand-authoring XML or significant investment in professional conversion tools or outsourced vendors — a barrier that falls hardest on smaller publishers and journals.
- **"Valid" is not "good."** Valid XML can still be functionally broken: a misplaced hyphen in a DOI can cause failed deposits in Crossref or PubMed Central, and insufficiently tagged references hinder discoverability. Validity guarantees conformance to a schema, not fitness for purpose.
- **Tag-set divergence.** Even though JATS is widely adopted, publishers frequently maintain their own subset conventions and house markup schemes. This "flexibility" makes authoring to a single compliant target difficult and undermines true interoperability.

3.2. The computational sticking points

The deeper problem for computational science is structural. The work of the *Notebooks Now!* project [12] and the companion "Science Communication with Notebooks" recommendations identified, concretely, what breaks when one tries to bring a computational notebook into the version of record [13]:

1. **Executable content has no native home.** JATS can hold code as text, but it has no first-class model for an executable cell, its kernel/language, its outputs, and the linkage between them.
2. **Outputs are flattened.** Interactive figures (e.g. Plotly [14], Bokeh [15], Altair [16]), 3D visualizations, and large-scale data views are reduced to static images. A one-and-a-half-terabyte microscopy image becomes a screenshot.
3. **Provenance and dependency are lost.** The relationship between a result, the code that produced it, the data it consumed, and the environment it ran in is not represented; it survives, if at all, in a supplementary ZIP file.
4. **The container, not the component, is the unit of credit.** Review, citation, and attribution attach to the article DOI, not to the figure, dataset, or method that other researchers may want to reuse.

The *Notebooks Now!* working groups concluded that **integrating computational documents into scholarly publishing requires a re-imagination of the publishing processes — from submission through peer review to production — and authoring tools that can execute content, display computational and interactive outputs directly, and archive that content faithfully.** OXA is an attempt to provide the missing data substrate for exactly that re-imagination.

3.3. San Diego standards meeting

OXA emerged at an in-person working meeting held in San Diego in November 2025, hosted by openRxiv and the Continuous Science Foundation and documented in the report "From Tools to Adoption: A Path to Modular and Interactive Scientific Publishing" [17]. Twenty-five open-science leaders — developers of the leading modular publishing tools, alongside licensing and metadata experts — gathered to build a working, federated reference architecture using real bioRxiv content and connected authoring tools. The convened tool developers spanned Stencila [18], MyST [3], Quarto [19], Curvenote [4], OpenAlex, and eLife. The explicit philosophy was implementation-first: prioritizing working software and demonstrable interoperability over abstract standardization.

In this meeting, a 'bedrock, soil, flowers' framework was put forth. In a modular science ecosystem, the modular elements of science - the data, code, images, protocols - are the 'bedrock'. They are the foundation on which scientific claims are built. The 'flowers' are the output of combining these modular elements - the articles, visualizations, discovery tools, the things we typically consume or explore as readers of science. What's missing now is not a lack of bedrock elements, or ideas for flowers and how science can be shared, but the connections from the bedrock that would allow for more 'flowers' and experiments in scientific communication. The 'soil' is the missing middle that provides the real basis for new models of scientific communication, attribution, discovery and remixing to occur. By working on that missing middle 'soil' layer, that's the unlocking and enabling function that makes so many more approaches possible, and what the tool developers at this meeting identified as a fundamental missing component in the ecosystem. It's crucial that these soil layers be community stewarded and governed, as an element of the ecosystem that is meant to connect different substrates and enable broad use and creativity from the developers of tools in the scientific space. This aligned with the gaps identified in the *Notebooks Now!* project and pointed again to the need for a connecting, implementation-focused standard

that is purpose-built and community-developed for sharing science digitally (where there is no need for the paper-shaped box) in a way that the modular elements of science in the 'bedrock' can be reused and remixed by both humans and machines. Therefore in Day 2 of this meeting as a collaborative group, we began the work on OXA with the defining community and technical principles outlined below.

4. The RFC and community process

OXA is explicitly designed as an open community specification rather than a single organization's product or platform. It is stewarded by the Continuous Science Foundation (CSF), a Canadian non-profit that convenes researchers, tool builders, publishers, and standards groups around modular, continuous publishing.

Governance proceeds through a Request for Comments (RFC) process: the technical structure is proposed, input is gathered from the community, and implementation is driven in parallel. The work is distributed across a set of public repositories in the `oxa-dev` GitHub organization.

The specification and schemas are CC0-licensed, and the tooling is MIT-licensed, lowering the barrier for tool builders, publishers, and repositories to adopt and extend the format. This community-first approach is central: the architecture rests on open standards and open-source values rather than on any single vendor's platform.

5. Technical architecture

OXA is a specification for representing scientific documents and their components as structured JSON objects. The format follows a typed node model with `children` arrays that form a tree, inspired by UnifiedJS [20] and the Pandoc AST.

Every OXA node shares a common shape with a small set of well-defined properties:

Table 2. *The common shape of an OXA node.*

Property	Type	Description
`type`	string (capitalized)	The node type, e.g. `Paragraph`, `Text`, `Heading`.
`id`	string	Unique identifier for referencing and linking nodes.
`data`	object	Arbitrary metadata (attributes, provenance, DOI).
`children`	array	Nested content nodes — block or inline types.

Node types divide into inline nodes (`Text`, `Emphasis`, `Strong`, `InlineMath`, `Link`, `Image`, `Cite`, `CiteGroup`, and others) and block nodes (`Paragraph`, `Heading`, `CodeBlock`, `BlockQuote`, `Section`, `Div`). A document is rooted in an `OXA` node carrying version and metadata (license, authors, title). A simple example of a paragraph OXA node is shown in Program 1 to demonstrate the nested structure of the node model. For more complex examples, see the OXA documentation and existing node schemas.

```yaml
type: Paragraph
children:
  - type: Text
    value: "This is "
  - type: Strong
    children:
      - type: Text
        value: "bold and "
      - type: Emphasis
        children:
          - type: Text
            value: "italic"
  - type: Text
    value: " text."
```

Program 1. *A very simple example of a paragraph OXA node in a document with some inline elements for bold, italic, and text. The markdown equivalent is:* `This is **bold and _italic_** text.`*. The JSON is shown in YAML format for readability.*

The design principles are:

- **Open by design** — JSON Schema (Draft-07) based and CC0-licensed.
- **Composable** — each node is self-contained, typed, and nestable or reusable.
- **Interoperable** — compatible with JATS, MyST Markdown, Stencila, Curvenote, and Quarto.
- **Extensible** — new node types can be added while preserving schema validation.
- **Typed and linked** — everything has a clear `type`, optional `id`, and structured `data`.
- **Modular** — documents and components can link across projects, enabling cross-references, citations, and reuse of figures, data, or methods from distributed sources.
- **Computational** — the schema builds in support for computational content such as notebooks.

OXA positions itself relative to its conceptual ancestors as a modernization for web-native use: JATS and the Pandoc AST as structural ancestors, Schema.org for naming concepts, Stencila as a functional and conceptual ancestor for computational typing, and Quarto, MyST Markdown and Curvenote as integrations to computational ecosystems. The architecture also includes packages for validation, rendering, and transformation that allow lightweight, independent implementations, and a conversion path to the AT Protocol (ATProto) `pub.oxa.*` lexicon namespace (see the ATProto OXA documentation for more details) — letting documents live in any Personal Data Server so that scientific content can become user-owned, portable, and discoverable alongside social feeds and moderation infrastructure. For example, Program 2 shows the ATProto representation of the same example as Program 1 in AT Protocol format.

```
{
  "$type": "pub.oxa.blocks.defs#paragraph",
  "text": "This is bold and italic text.",
  "facets": [
    {
      "index": { "byteStart": 8, "byteEnd": 23 },
      "features": [
        { "$type": "pub.oxa.richtext.facet#strong" },
        { "$type": "pub.leaflet.richtext.facet#bold" }
      ]
    },
    {
      "index": { "byteStart": 17, "byteEnd": 23 },
      "features": [
        { "$type": "pub.oxa.richtext.facet#emphasis" },
        { "$type": "pub.leaflet.richtext.facet#italic" }
      ]
    }
  ]
}
```

Program 2. *The AT Protocol representation of OXA uses "facets" instead of a tree. The text is stored as a single plain string, and formatting is described by byte-range annotations. The conversion tools* `oxa convert --to atproto examples/document.json` *will convert an OXA document to this format and include other common facet annotations used by the community such as Leaflet (https://leaflet.pub). The code shows the same example as Program 1 in AT Protocol format.*

6. Provenance and lineage of OXA

OXA's design synthesizes lessons from several lines of prior work: JATS as the structural ancestor of a standardized scholarly record; the Stencila schema for typed, executable documents; Curvenote's work in scholarly publishing and interactive articles; and the document-transformation pipelines of Pandoc, MyST Markdown, UnifiedJS [20], and Quarto.

JATS is the structural starting point. From it, OXA borrows the core idea of a single, standardized, machine-readable vocabulary for scholarly content, and inherits much of its element naming. What OXA does not inherit is JATS's print-shaped orientation. This limitation was recognized independently by many of the projects in this space.

The most direct ancestor is the Stencila schema. Since 2019, Stencila has represented executable documents as a tree of JSON data validated by JSON Schema, with the explicit aim of providing a framework for highly structured, strongly typed executable documents. The Stencila schema extends Schema.org types — adopting `ScholarlyArticle`, `Dataset`, `Person`, and `Organization` where they exist, and adding executable extensions such as `CodeChunk` and `Parameter` where Schema.org falls short. This canonical model is the source of truth across Stencila's parsers, codecs, kernels, and publishing pipelines, allowing documents to round-trip between formats. OXA inherits this typed-node, Schema.org-aligned philosophy directly, and Stencila's founder, Nokome Bentley, is a co-author of this paper and a member of the OXA steering council.

A parallel thread runs through Curvenote's work in interactive scholarly publishing, which fed directly into the broader MyST ecosystem described below. In the American Geophysical Union (AGU) Notebooks Now! project — an effort to develop an end-to-end workflow for submitting, peer-reviewing, and publishing computational notebooks as a primary element of the scientific record [12] — Curvenote and Posit (the makers of Quarto) collaborated within the working groups, prototyping how notebooks could move through submission, review, and production. The findings of that work are documented in the *Notebooks in Publishing* report [13].

Both the Stencila schema and the Notebooks Now! working groups arrived at the same conclusion: JATS cannot accommodate computational content shoehorned in after the fact, and its print-shaped model is a functional blocker to interactive or more modular ways of working with scientific content. The integrations that modern research demands, including executable cells, live/interactive outputs, component-level provenance, and modularity, have no native home in a format designed to describe static, print documents.

The way past that blocker is a structured intermediate representation that accommodates computational content and supports modularity and composability. There are now several demonstrations of this working at scale. Pandoc popularized the pattern: parse documents into a typed abstract syntax tree and transform that tree into many standardized outputs [21]. Quarto [19] and MyST Markdown build on the same idea for computational narratives. MyST (Markedly Structured Text) is a community-driven superset of CommonMark with first-class support for citations, cross-references, executable directives, and scholarly metadata (researcher identifiers (e.g. https://orcid.org), funding information, or contributor roles (e.g. https://credit.niso.org/)) [3]; its command-line tooling began at Curvenote as the `curvenote` CLI, became `mystmd` in 2023, and is now part of Project Jupyter [22]. The MyST Document Engine produces a MyST AST that preserves narrative content, computational blocks, outputs, citations, figures, and metadata. Both MyST[1] and Quarto are deployed across thousands of projects [3], demonstrating that AST-driven, computational narratives can work across many domains and output formats at scale.

7. Why a new standard

Every mention of a new standard makes one think of the xkcd comic on standards. That comic proceeds as **Panel 1:** 'Situation: there are 14 competing standards'. **Panel 2:** '14?! Ridiculous! We need to develop one universal standard that covers everyone's use cases'. **Panel 3:** 'Soon: Situation: there are 15 competing standards.' The motivation for OXA is actually in opposition to Panel 2. There are existing generalized standards or approaches in prior works that do an excellent job of covering a wide range of use cases. There are also existing standards like JATS and others specifically for a print version of scientific communication. However, to create a purpose-built standard for our current digital era, with both humans and machines as users of this standard, we need something focused on these needs, rather than trying to build in niche needs to a general solution or repurpose a not-right-sized existing standard. Creating a new approach **that can connect to any of the existing standards** lets us build on where we are without needing users, infrastructure organizations or tool developers to change.

OXA generalizes efforts into a shared *exchange* format. Where for example, MyST and Quarto are authoring syntaxes and document pipelines, OXA is a neutral, web-native format onto which all of them — along with JATS — can map.

We also, as a scientific communication community, need a framework and coordination approach and a community-governed and neutral space, that allows us to engage together in the process, staying aligned with the goals for scientific sharing without needing to create overly general solutions or be beholden to the needs of a specific few. This is again borrowing from the approaches of the open source community, with Requests for Comments, contributor guidelines, version control and transparent communication and decision making. Therefore the governance structure of OXA is as important as the technical foundation in allowing for institutional adoption that can move us towards a standard that is adopted across millions of articles.

[1]There are over 18,000 dependencies of Jupyter Book, an implementation of MyST, based on GitHub dependency graph: https://github.com/jupyter-book/jupyter-book/network/dependents.

8. Related work: provenance, attribution, and research objects

While we see that OXA fills a crucial need and niche, it does enter a crowded landscape of efforts to make research machine-readable and reusable. OXA is meant to complement, learn from and connect with existing efforts and these projects continue to be crucial in the ecosystem. We review the most relevant active community projects, emphasizing community-led work.

JATS and JATS4R. JATS remains the substrate of the published scientific record, and JATS4R [23] provides community recommendations to make JATS tagging more consistent and reusable. OXA does not seek to replace JATS as an archival format so much as to provide a web-native, computation-aware layer that can interoperate with it — as demonstrated by the bioRxiv translation described below.

MECA. The Manuscript Exchange Common Approach (MECA), a NISO Recommended Practice, defines a packaging format for moving manuscripts between submission systems, preprint servers, and production services [24]. MECA is the closest existing analogue to OXA's exchange goal, however, it inherits JATS's static-document assumptions: it packages finished manuscripts in a ZIP file rather than modeling living, computational, component-level content. OXA can be read as a "modern MECA+JATS" alternative that replaces the static-document assumptions embedded in JATS with a more modern, computational-aware approach that can be accessed in a web-native way.

RO-Crate and Research Objects. RO-Crate is an open, community-driven, lightweight approach to packaging research artifacts with their metadata, based on Schema.org annotations in JSON-LD [25]. OXA and RO-Crate are complementary: RO-Crate excels at describing and bundling a *collection* of artifacts and their metadata, while OXA models the *internal structure* of the document and its components as a typed tree. An OXA document could be packaged within an RO-Crate.

Reproducible packaging: Whole Tale, Binder. Whole Tale [26] captures the whole computational environment ("tale") needed to reproduce an analysis, and Binder/BinderHub [27] turn a code repository into an executable, interactive environment in the browser. These projects solve the *execution and environment* problem that complements OXA's *representation* problem.

A web of overlapping collaborations connects these efforts: eLife and Stencila on Executable Research Articles [28]; AGU and Curvenote on Notebooks Now! [12]; and openRxiv with Curvenote on the Reader implementation described next [29]. OXA is best understood as the point where these threads converge, coordinated through CSF's working groups and the RFC process.

9. First large-scale implementations

9.1. *Curvenote* × *openRxiv*

The first large-scale implementation of OXA is a partnership between Curvenote (https://curvenote.com) and openRxiv (https://openrxiv.org), the independent non-profit that stewards bioRxiv and medRxiv. On June 11, 2026, openRxiv launched its first openRxiv Labs experiment: the Curvenote Reader, an interactive reading experience layered over the entire bioRxiv corpus [29].

The enabling work was a translation of openRxiv's JATS-format XML archive into an early version of OXA. The Reader lets readers explore references, terminology, expanded figures, and related works while staying in the context of the original preprint, with the same URL structure as bioRxiv. The translation covered 41.7TB and over 500,000 preprint versions in

the bioRxiv and medRxiv corpus. The translation pipeline is supported using the `jats` project (https://github.com/continuous-foundation/jats), which has been tailored by Curvenote to the specific needs of the openRxiv corpus.

This implementation is significant for three reasons. First, it demonstrates that a large, real-world JATS archive can be translated into OXA at scale. Second, it validates OXA's "soil" thesis: once content is structured, new "flower" experiences (interactive reading, querying, future tool development) can be built on top. Third, it embodies the constellation model of research that openRxiv has articulated: where a preprint is one node connected to data repositories, reviews, replications, and trust signals across multiple organizations rather than a self-contained PDF. OXA is the structural substrate that makes such a constellation addressable. For example, Figure 1 shows a hover-citation that uses the OXA schema to display a citation with accompanying metadata as well as figure previews. The figure content is pulled directly from a selection of nodes in the OXA document. These content previews could be extended to AI-chat interfaces and other interactive reading experiences.

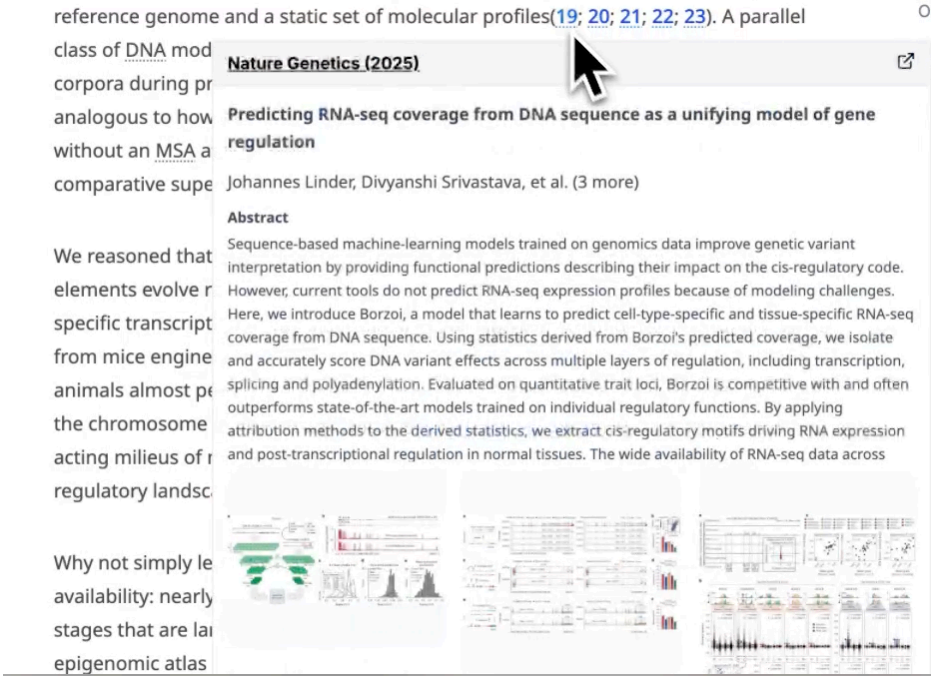


Figure 1. *A hover-citation using Curvenote Reader that uses the OXA schema and translated documents to display a citation with accompanying metadata as well as figure previews. The figure content is pulled directly from a selection of nodes in the OXA document.*

9.2. eLife Pathways × Stencila

eLife Pathways (https://elifepathways.org) is another organizational implementation of OXA through an expanding collaboration with Stencila (https://stencila.io) and adapting eLife Pathways open-source Enhanced Preprint Platform to natively support OXA. There are also plans to build OXA support with Kotahi (https://github.com/elifepathways/kotahi). These integrations create new opportunities for collaboration between technology and publishing projects within the broader, interoperable scholarly communications ecosystem.

10. Emerging tooling

A small ecosystem of tooling is beginning to form around the specification, spanning both validation and conversion.

The canonical reference tool is the `oxa` command-line utility, developed in the oxa-dev organization. The scope of the CLI is deliberately narrow: it validates OXA documents against the published schemas. It can check a single file or a whole glob, read from standard input for use in pipes, validate a fragment against a specific node type (for example, `oxa validate --type Heading`), and report results through conventional exit codes so that it slots cleanly into continuous-integration and pre-commit checks. It fronts a small package family that includes `@oxa/core` for programmatic validation and `oxa-types` for TypeScript definitions and `@oxa/react` for a minimal demonstration of rendering.

```
# Install the OXA CLI tool
pip install oxa

# Validate a JSON file
oxa validate document.json

# Validate multiple files
oxa validate *.json

# Validate from stdin
cat document.json | oxa validate -

# Validate YAML files
oxa validate document.yaml

# Validate against a specific node type
oxa validate --type Heading heading.json
```

Program 3. *The OXA CLI tool allows you to validate OXA documents from the command line:*

The OXA CLI tool is a wrapper around the OXA programmatic APIs, which allow you to validate OXA documents from Python, JavaScript, or Rust. For example, Program 4 shows how to validate an OXA document from JavaScript. The `oxa-types` library is available on `pip`, `npm`, and `cargo` and the JSON Schema is available at https://oxa.dev/v0.2.1/schema.json.

```
import { validate } from "@oxa/core";
import type { Document } from "oxa-types";

const document: Document = {
  type: "Document",
  children: [
    {
      type: "Paragraph",
      children: [{ type: "Text", value: "Hello, world!" }],
    },
  ],
};

const result = validate(document);
if (result.valid) {
  console.log("Document is valid!");
} else {
  console.error("Validation errors:", result.errors);
}
```

Program 4. *The OXA programmatic APIs allow you to validate OXA documents from JavaScript.*

Conversion is the second strand, and the most developed and accessible work to date comes from Stencila. Building on its canonical, strongly typed schema, Stencila has implemented a bidirectional OXA codec that both decodes OXA JSON into the Stencila document model and encodes that model back out to OXA, tracking any information lost in either direction. Because this conversion routes through the same canonical schema that already backs Stencila's parsers and codecs for many other formats, it inherits that format support: MyST Markdown, Quarto (`.qmd`), Jupyter notebooks (`.ipynb`), Microsoft Word (`.docx`), and other Markdown flavors can be converted into OXA — and back out again. **This is the practical**

payoff of the typed-node lineage traced above: rather than asking authors to adopt a new syntax, OXA can meet them in the formats they already use, and documents they have already written become a source of structured, OXA-native content.

11. Open standards as the substrate

The open standards stack we view as load-bearing for modular science is, in our reading: a structured document format with executable extensions; persistent, component-level identifiers for figures, datasets, code artifacts, and computational environments — not a DOI for every sentence, but a high-fidelity *linking system* inside the bundle that carries licensing, attribution, and other metadata; an open preprint foundation (e.g. the broader openRxiv ecosystem) that accepts and serves modular outputs as first-class; machine-legible trust signals; and reproducible execution environments. Every one of these has working open-source implementations today. The gap is integrative and connective: producing tooling that hides this complexity from the researcher while preserving the openness of the substrate beneath.

Cultures change when the new way is *better* than the old way, offering a 'moving towards', rather than just a 'moving away from'. The most durable path to a modular research record is to make modular outputs visibly more useful than the PDF — to the original authors, their collaborators, and the broader field — and to make the path from existing work to modular output smooth enough that a typical lab can take it easily. Funder and publisher policies and changing incentive structures are still crucial, but when there is a real carrot with a compelling reason to change, those policies can be more effective and impactful.

12. Governance and steering

OXA is stewarded by the Continuous Science Foundation. Day-to-day technical direction rests with a steering council whose current members are: Nokome Bentley (Stencila), Rowan Cockett (Curvenote), and Tracy Teal (openRxiv). The steering council operates within CSF's broader governance, and changes to the specification proceed through the open RFC process, ensuring a diversity of perspectives and that no single organization controls the direction of the standard.

13. Conclusion and future work

OXA is an early-stage but already-implemented attempt to give modern, computational, modular science a native exchange format. By representing documents and their components as typed, addressable, CC0-licensed JSON objects, it aims to unbundle research from its paper-shaped legacy and expose the evidence underneath — making figures, notebooks, data, and methods first-class, reusable, attributable, and machine-actionable. Its provenance in the Stencila schema, Curvenote, and MyST Markdown gives it a running start, and its first large-scale deployment over the bioRxiv and medRxiv corpus demonstrates feasibility at scale.

Much remains to be done. The schema is at an early version and will evolve through the RFC process; the licensing-and-attribution framework for modular components is under active development; and the 2026 roadmap includes further large-scale pilots and deeper integration of the Reader experience into bioRxiv and medRxiv. The broader test will be adoption: whether tool builders, publishers, and repositories converge on OXA as a shared layer. A standard is only as useful as the number of tools and institutions that actually emit and consume it.

Other limitations or things to continue to address include how the OXA structuring occurs, "AI-readability" and the handling of versioning and continuous content. Even with a friendlier format, producing well-typed, richly linked OXA documents takes good conversion tooling, and that tooling is still in its early stages and designed for particular groups. Creating the format needs to be achievable for everyone, not just well-resourced or particularly tech-savvy players. Additionally the current work has focused on the use case of 'flowers' designed for humans, like new reading experiences. We need to test against use cases with AI agents and LLMs that are increasingly the consumers of scientific communication to close that "AI-readability" gap. Finally, OXA leans into preprints and computational content that changes over time. It's a strength for continuous science, but it means that OXA needs to be designed to handle versioning and pinning, so that both modularity and provenance are maintained. Directed work and community engagement around these topics will be crucial.

We invite the SciPy community — long a leader in computational, open, and reproducible science, and the originators of much of the composable software ecosystem this paper takes as its model — to engage with the specification, contribute RFCs, and build on the architecture to ensure it's fit for purpose and for the future.

References

[1] S. Chen, T. Alrashed, A. Halevy, and N. Noy, "Do Data Agents Need Semantic Metadata? A Comparative Study in Agentic Data Retrieval." [Online]. Available: https://arxiv.org/abs/2605.28787

[2] Q. Deng *et al.*, "Unstructured Data Analysis using LLMs: A Comprehensive Benchmark." [Online]. Available: https://arxiv.org/abs/2510.27119

[3] P. Jupyter *et al.*, "Jupyter Book 2 and the MyST Document Stack: A modular, extensible, web-native stack for authoring and publishing computational narratives," in *Proceedings of the 24th Python in Science Conference*, in SciPy. 2025, pp. 173–193. doi: 10.25080/hwcj9957.

[4] R. Cockett, S. Purves, F. Koch, and M. Morrison, "Continuous Tools for Scientific Publishing: Using MyST Markdown and Curvenote to encourage continuous science practices," in *Proceedings of the 23rd Python in Science Conference*, in SciPy. 2024, pp. 121–136. doi: 10.25080/nkvc9349.

[5] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[6] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[7] The Pandas Development Team, "pandas-dev/pandas: Pandas." [Online]. Available: https://doi.org/10.5281/zenodo.3509134

[8] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman, Eds., 2010, pp. 56–61. doi: https://doi.org/10.25080/Majora-92bf1922-00a.

[9] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: https://doi.org/10.1109/MCSE.2007.55.

[10] National Information Standards Organization, "JATS: Journal Article Tag Suite, version 1.4 (ANSI/NISO Z39.96-2024)." [Online]. Available: https://jats.nlm.nih.gov/

[11] D. Spinellis, "Open reproducible scientometric research with Alexandria3k," *PLOS ONE*, vol. 18, no. 11, p. e294946, 2023, doi: 10.1371/journal.pone.0294946.

[12] G. Caprarelli, B. Sedora, M. Ricci, S. Stall, and M. Giampoala, "\textit{Notebooks Now!} The Future of Reproducible Research," *Earth and Space Science*, vol. 10, no. 12, 2023, doi: 10.1029/2023ea003458.

[13] Curvenote and AGU Notebooks Now! Working Groups, "Notebooks in Publishing." [Online]. Available: https://curvenote.github.io/notebooks-in-publishing/

[14] Plotly, "Plotly." [Online]. Available: https://plotly.com/

[15] Bokeh, "Bokeh." [Online]. Available: https://bokeh.org/

[16] Altair, "Altair." [Online]. Available: https://altair-viz.github.io/

[17] R. Cockett and T. Teal, "From Tools to Adoption: A Path to Modular and Interactive Scientific Publishing," 2025, doi: 10.62329/kcep6732.

[18] N. Bentley, "Stencila." [Online]. Available: https://stencila.io/

[19] J. Allaire, C. Teague, C. Scheidegger, Y. Xie, C. Dervieux, and G. Woodhull, “Quarto.” [Online]. Available: https://github.com/quarto-dev/quarto-cli

[20] unified.js, “unified.js: The unified compiler.” [Online]. Available: https://unifiedjs.com/

[21] J. MacFarlane, “Pandoc: a universal document converter.” [Online]. Available: https://pandoc.org/

[22] T. Kluyver *et al.*, “Jupyter Notebooks - a publishing format for reproducible computational workflows,” in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Scmidt, Eds., Netherlands, 2016, pp. 87–90. [Online]. Available: https://eprints.soton.ac.uk/403913/

[23] National Information Standards Organization, “JATS4R: Journal Article Tag Suite for Reuse.” [Online]. Available: https://jats4r.niso.org/

[24] National Information Standards Organization, “MECA: Manuscript Exchange Common Approach (NISO RP-30-2023),” techreport, 2023. [Online]. Available: https://www.niso.org/standards-committees/meca

[25] S. Soiland-Reyes *et al.*, “Packaging research artefacts with RO-Crate,” *Data Science*, vol. 5, no. 2, pp. 97–138, 2022, doi: 10.3233/ds-210053.

[26] A. Brinckman *et al.*, “Computing environments for reproducibility: Capturing the ``Whole Tale'',” *Future Generation Computer Systems*, vol. 94, pp. 854–867, 2019, doi: 10.1016/j.future.2017.12.029.

[27] P. Jupyter *et al.*, “Binder 2.0 - Reproducible, interactive, sharable environments for science at scale,” in *Proceedings of the 17th Python in Science Conference*, in SciPy. 2018, pp. 113–120. doi: 10.25080/majora-4af1f417-011.

[28] eLife, “eLife launches Executable Research Articles for publishing computationally reproducible results.” [Online]. Available: https://elifesciences.org/for-the-press/eb096af1/elife-launches-executable-research-articles-for-publishing-computationally-reproducible-results

[29] openRxiv and Curvenote, “openRxiv Labs Curvenote Reader.” [Online]. Available: https://reader.openrxivlabs.org/